

BADM500 - Implementation of a federated blogging platform using ActivityPub

Mohammad Adnan Amin (10/03/95) (moami14)

Karim Adnan Amin (01/04/04) (kaami23)

Supervisor: Stelios Tsampas

01/06/2026

Contents

1	Abstract	4
2	Introduction	4
3	The Fediverse, federation, ActivityStreams 2.0 and ActivityPub	5
3.1	The Fediverse	5
3.2	Federation	6
3.3	ActivityPub	6
4	Background and Related Work	8
4.1	The rise of Mastodon and the Fediverse	8
4.1.1	Twitter to Mastodon	9
4.1.2	Centralization in decentralized platforms	10
4.1.3	Toxicity	11
4.1.4	User Behavior After Migration	11
4.2	Challenges	12
4.2.1	Instances	12
4.2.2	Instance availability	12
4.2.3	Federation and user connectivity	14
4.2.4	Replication	15
5	Concepts	16
5.1	What is an endpoint?	16
5.2	What is an object?	16
5.3	Actor and collections	17
5.4	Inbox and Outbox	17
5.5	Activities	18

5.6	WebFinger	20
5.6.1	RFC 7033	21
5.7	Security and Keys	22
5.7.1	HTTP Signature	22
5.7.2	RSA and SH256	24
6	Implementation and Architecture	24
6.1	Project Structure	26
6.2	GET object endpoint	26
6.3	GET actor endpoint	27
6.4	GET collections	28
6.5	Follower model	28
6.6	WebFinger	29
6.6.1	How WebFinger works?	29
6.7	Deployment	30
6.8	Public Key	30
6.9	HTTP Signature	31
6.10	Inbox and Follow	32
6.11	Content Delivery to Followers	33
6.12	Incoming Undo	34
6.13	Incoming Create	34
6.14	Outgoing Follow	35
7	Architecture	36
7.1	Sequence diagram	36
7.2	C4 model	37
7.2.1	Context	37
7.2.2	Containers	38
7.2.3	Components	38
8	Testing	39
8.1	WebFinger URL Testing	39
8.2	WebFinger Mastodon Testing	40
8.2.1	HTTP Signature testing	41
9	Discussion	42
9.1	Django implementation	42
9.2	Challenges during implementation	43
9.2.1	Security mechanisms	44
9.3	Limitation and future work	44
9.3.1	HTTP Signature	45
9.3.2	Scalability	45
9.4	Implementation of Follower model	46

1 Abstract

This project presents the design and implementation of a web based blogging platform that is interoperable with the Fediverse through ActivityPub protocol [2]. The implementation is built using the Django web framework, along core ActivityPub features including Webfinger actor discovery, inbox and outbox handling, HTTP signature authentication, follower management, and federated content delivery.

The implementation was tested against Mastodon.social, where we can confirm that communication works with an established fediverse platform. The implementation demonstrates how Actor discovery, follow, content delivery, and incoming replies work.

The report discusses implementation challenges and known limitations, including missing HTTP Signature verification for incoming activities and synchronous content delivery without retry mechanisms.

2 Introduction

In today's world, traditional blogging platforms such as WordPress operate as isolated services with limited opportunities for interaction between independent platforms. Users who want to interact and follow content across different platforms usually need to create separate accounts for each service. This, of course, limits how far content can spread and creates more isolated communities.

The Fediverse offers an alternative approach through the ActivityPub protocol. Instead of requiring separate accounts when searching for new content, the protocol enables independent servers to communicate and exchange content without relying on a central server. One of the most widely known platforms in the Fediverse is Mastodon, which functions similarly to X/Twitter but uses ActivityPub protocol for federation. That means users will not have to create a separate account to communicate with others from a different server.

This project investigates how a web based blogging platform can be implemented to distribute content across the fediverse using the ActivityPub protocol. The project is built using the Django web framework and focuses on the core ActivityPub features needed for federation. This includes actor endpoints, WebFinger, inbox and outbox handling, HTTP signatures, and content delivery to followers.

Specifically, the research question is: *How can web based blogging platform be implemented to reliably distribute content across the fediverse using the ActivityPub protocol?*

3 The Fediverse, federation, ActivityStreams 2.0 and ActivityPub

3.1 The Fediverse

We are all familiar with the classic social media platforms such as YouTube, Twitter, Instagram, Facebook, and Reddit. One thing they all have in common is that they are centralized platforms with one single company controlling all the servers, moderation, and user data. In contrast, the Fediverse represents a decentralized approach to social media. Instead of relying on a central authority, the Fediverse is a large network that consists of independently operated platforms and servers.

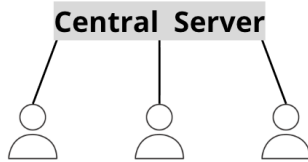
Platforms refer to the many different types of service that exist within the Fediverse. For example, we got video and media sharing platforms like Peertube and Pixelfed, which are similar to YouTube and Instagram, Mastodon, which functions similarly to X, and Lemmy, which functions similarly to Reddit.

Although platforms are operated separately, their servers, which are usually referred to as instances, can still communicate with each other through federation, meaning that it should not matter which decentralized platform you are using or which server a user is on, they are still able to interact with other users, instances, and platforms without necessarily needing an account on every independent service. When we say "interact", we mean actions such as following other users, replying to or liking content, sharing posts, and viewing content through interoperability across federated platforms.

The Fediverse gives users and communities more independence and ownership, since anyone can operate their own server, define their own rules regarding moderation or what the server should do/be, and manage their own community independently, while still remaining connected to the rest of the decentralized network.

A simplified illustration of how centralized and decentralized architecture is shown below:

Centralized Architecture



Federated Architecture

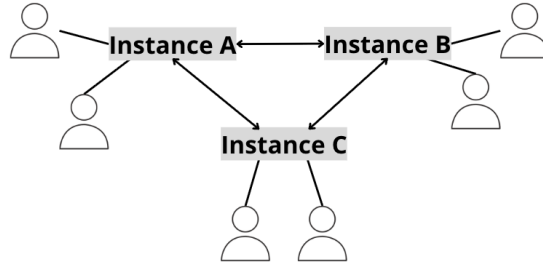


Figure 1: Illustration of centralized and decentralized architecture

3.2 Federation

To fully understand the Fediverse, we need to become familiar with the concept of federation”. Federation can be compared to telephone networks, where users may have different phone providers, but are still able to communicate because all providers follow the same communication protocols. Similarly, servers and platforms in the Fediverse are independently operated, but can still exchange activities and data through shared protocol such as ActivityPub

3.3 ActivityPub

To support federation within the Fediverse, we would need a communication protocol to enable decentralized servers to interact. This is where the ActivityPub protocol becomes relevant. ActivityPub is the protocol used to connect everything and support communication between different independent servers.

ActivityPub distinguishes between two different ways of interaction, client to server and server to server communication. The client to server communication allows a client application, such as a mobile application or web application, to interact with a user’s own server. In other words, it is communication between the same user and their own server, just through a client.

The server to server communication is what we are implementing, and it allows communication between different federated servers. It allows servers to interact with activities such as posts, likes, and follows.

An example of a client to server would be a mobile application connecting to its own server, and creating a post on behalf of the same user. In contrast, server to server communication happens when one server delivers an activity, such as a post or a like, directly to another remote server that is part of the federation.

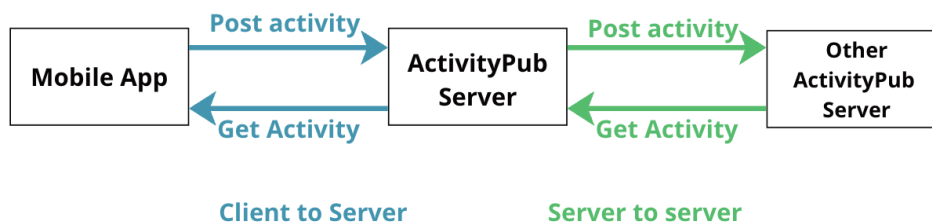


Figure 2: ActivityPub client to server and server to server communication

ActivityPub uses ActivityStreams 2.0[1] for communication, which can now make the architecture somewhat confusing. ActivityStreams 2.0 is referred to as the vocabulary of ActivityPub, it can be seen as a phonebook that defines different types of objects and activities that the federated servers understand.

For example, ActivityStreams 2.0[22] defines different objects, such as **Note**, **Video**, and **Person**, and **Mention**, it also defines activities as **Create**, **Like**, **Accept**, **Delete**, **Follow**, and many more. Together, these definitions shape how the federation works.

ActivityPub is built on top of this vocabulary and defines how these objects and activities are used for interactions between federated servers. In this sense, ActivityStreams 2.0 defines what is being communicated, while ActivityPub defines how the communication happens between federated servers. ActivityPub also has actors that are an essential part of the protocol. An actor is any entity, such as an application, users(referred to as people), or services that can interact with these activities. actors are described in more detail later in the project.

The protocol uses a version of JSON[12] for sharing data and communication called JSON LD(JavaScript Object Notation for Linked Data), where in ActivityPub, Linked Data refers to an extra field added to standard JSON documents. This makes it possible for other different systems to connect and share data in a way that can be understood. This is done through the **@context** field, which links the document to the ActivityStreams 2.0 vocabulary, and as a result, different servers across the Fediverse can now understand the different activities and objects, such as **Create**, **Undo**, **Follow**, **Person**, **Note**, in a standardized way.

It's important to note that ActivityPub is not the only protocol that allows the Fediverse to exist or allows decentralized activities to take place. Other protocols, such as OStatus, have also been used, which was previously used by Mastodon before transitioning to ActivityPub. AT Protocol and Diaspora are also alternatives for decentralization. However, ActivityPub has become one of the most adopted protocols, especially since popular decentralized platforms have started to adopt it.

Another important distinction is that decentralized protocols do not nec-

essarily have the ability to communicate with each other. For example, BlueSky functions similarly to Mastodon, but it utilizes the AT Protocol instead of ActivityPub. As a result, even though both platforms support decentralization, users on Bluesky cannot directly interact with users on Mastodon or other ActivityPub platforms.

4 Background and Related Work

Mastodon is one of the most popular decentralized micro-blogging (we call it note blogging) platforms released in 2016. It was released as an alternative to the centralized Twitter. It is free and open source, meaning that the software can be used and modified by anyone.

Unlike traditional social media platforms, Mastodon is not controlled by one single company or server. Instead, it consists of many independently managed servers, called instances. Users create an account on one instance, but they can still communicate with users on other instances through federation.

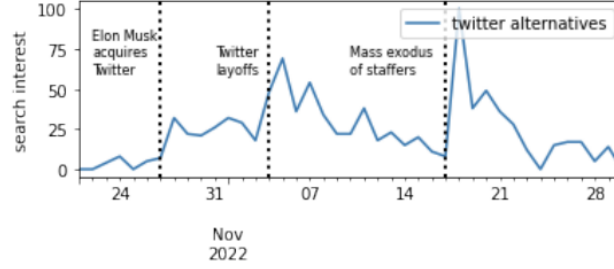
As already mentioned, this is made possible with the ActivityPub protocol, and Mastodon is a very important part of the Fediverse, since it's one of the most widely known decentralized social media platforms.

4.1 The rise of Mastodon and the Fediverse

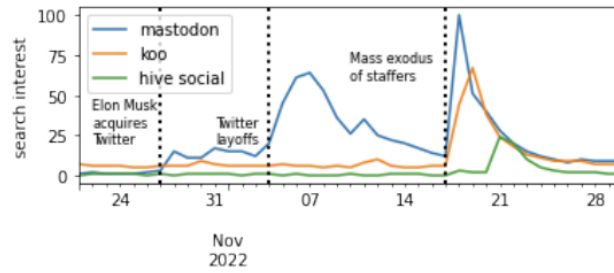
In 2022, Elon Musk managed to buy Twitter[23], which caused doubt and controversy among many users of the platform. Concerns regarding moderation, control of the platform, and changes in leadership led many users to search for alternatives to traditional centralized social media platforms. Suddenly, decentralized social media platforms gained more attention, especially platforms that offered similarities to already existing platforms.

Some of these platforms were Mastodon, which is a decentralized blogging platform that functions similarly to Twitter. Another example is PeerTube, which is a decentralized video-sharing platform that provides functionality similar to YouTube while relying on federation between independent servers.

The article, "Flocking to Mastodon" [23], presents the following figure:



(a)



(b)

Figure 3: Search interest of Twitter alternatives

The figure(a) shows the search interest for alternatives to Twitter in 2022. As shown, the interest in other platforms increased significantly, especially during the period of layoffs at Twitter and later during the resignation of staff. Figure(b) shows how Mastodon was one of the most popular alternatives compared to platforms such as Koo and Hive.

4.1.1 Twitter to Mastodon

The study "Flocking to Mastodon" goes through the migration of users from Twitter to Mastodon following Elon Musk's purchase of Twitter in 2022. The study started collecting data to analyse both the growing interest in Mastodon and how many users actually migrated from Twitter to the decentralized platform.

To do this, the study collected tweets from Twitter that contained Mastodon-related keywords, such as "bye bye twitter" and "mastodon", as well as hashtags like "MastodonMigration" and "GoodByeTwitter". They also collected links to Mastodon instances shared on Twitter.

The study then matched Twitter users with Mastodon accounts in order to identify users who had migrated between the two platforms. Using this approach, the study identified 136009 Twitter users who migrated to 2879 different Mastodon instances. The results also showed that even established

and verified users migrated.

4.1.2 Centralization in decentralized platforms

Despite Mastodon being designed as a decentralized platform, it is facing some centralization challenges. The study found that many migrated users gathered around a small number of large instances instead of spreading more across the network, especially <https://mastodon.social/>, which is funny enough, as it is also the instance used in our project.

This is interesting because it shows that even though the technology and architecture are decentralized, this does not automatically guarantee decentralization in practice. Even if anyone can create and run their own Mastodon instance, users can still prefer the most well-known server to join. The following figure shows the 30 different Mastodon instances that Twitter users moved to the most.

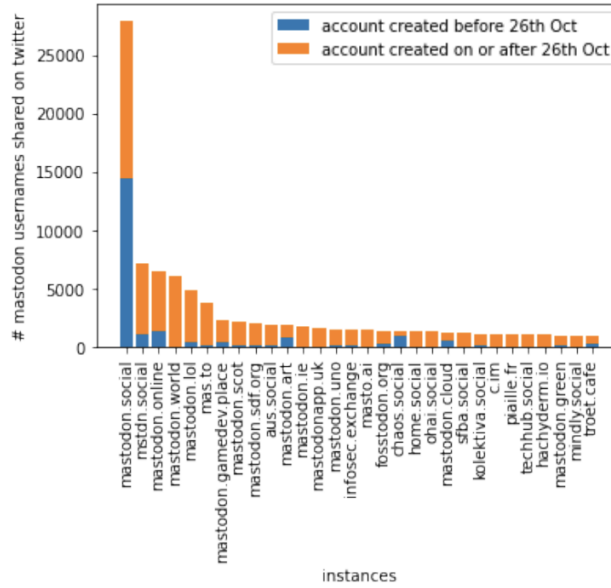


Figure 4: Mastodon instances Twitter users moved to

One explanation mentioned in the study is that users tend to join larger and more mainstream instances in order to build larger social networks and communities quickly. Another possible explanation is that new users may find large instances safer to join, since they appear more active and established compared to smaller instances.

It is also possible that many users are not fully familiar with the idea of decentralized platforms. New users who come from centralized platforms such as Twitter may naturally look for the largest Mastodon instance be-

cause this feels similar to joining one main platform. In this way, even a decentralized system can experience centralization through user behavior.

4.1.3 Toxicity

One of the challenges faced on Mastodon and other decentralized platforms is moderation and toxicity, which the study also investigates. Since Mastodon is decentralized, moderation is handled by individual instance administrators instead of a single centralized moderation team, like on Twitter. This can create challenges, as smaller instances may not have enough resources to handle toxic or harmful content.

The study measured the toxicity of posts from migrated users on both Mastodon and Twitter. The results ended up showing that Twitter was the most toxic, where 5.49% of the tweets were categorized as toxic, compared to 2.80% of the posts on Mastodon. However, moderation is usually handled by volunteers, which may make it difficult to maintain low levels of toxicity and harmful content in the long term.

4.1.4 User Behavior After Migration

The study also analyzed how users behaved after migrating to Mastodon. For example, about 1.53% of users on both Twitter and Mastodon had identical posts, meaning that the users may have used the two platforms differently instead of posting the same content.

This is interesting, as you might think that users would try to post similar content on both platforms in order to maintain their communities and make the transition from Twitter to Mastodon easier. However, the results suggest that users may have viewed Mastodon as a different type of community and culture compared to Twitter. Content and behavior that were common on Twitter may not necessarily have been as relevant on Mastodon.

The study also found more activity in smaller Mastodon instances. In particular, smaller instances had around 121% more posts compared to larger instances. One possible explanation is that users on smaller instances were interested in the ideas behind decentralization and federation, and became more engaged with the platform and its communities.

Another explanation is that the migration to Mastodon may have been part of a trend for some users during the controversy surrounding Twitter in 2022. It might have been the "cool thing" to do at that time, and naturally many users joined <https://mastodon.social/> because it was the most popular instance. In contrast, users on smaller instances may have had a greater interest in decentralized platforms and the ideas behind federation and independent servers. This could also explain why users on smaller instances appeared to be more active and engaged compared to users on the larger instances.

4.2 Challenges

Another interesting study regarding Mastodon and decentralized platforms is "Challenges in the Decentralised Web: The Mastodon Case"[19]. The study conducts an analysis of Mastodon and its activities over a period of 1 year and 3 months, analyzing around 67 million posts (referred as "toots" in the article). The study focuses on Mastodon to find out how decentralized social platforms work in practice.

The dataset covers three parts: Mastodon instances, post content and information about how users follow each other across different instances. In total, the study includes data from 4328 different instances, and 67 million public posts.

4.2.1 Instances

Between 2017 and 2018, Mastodon experienced significant growth in both users and active servers. The researchers suggest that some of this growth, especially during early 2018, may have been connected to criticism of large social media platforms such as Facebook. The article does not claim this as a certain cause, but suggests that events such as the "#DeleteFacebook" campaign, together with more political involvement around Facebook, may have encouraged users to explore decentralized alternatives that focused more on privacy and user control.

The study separates Mastodon instances into two different registration types. Slightly more than half of the servers were closed, meaning users needed approval or an invitation before they could create an account. The remaining servers allowed open registration, where any user could join freely. Open servers generally attracted more users, but the closed communities showed higher activity per user on average. This suggests that smaller and more controlled communities often create stronger engagement between users.

Another important observation from the study is that Mastodon still shows signs of centralization, even though the platform is designed to be decentralized. A relatively small number of instances hosted a very large percentage of the users. Approx. the top 10% of servers covered almost half of all registered users. This mirrors patterns that are also seen on centralized social media platforms and raises questions about whether decentralization alone is enough to avoid concentration around a few large servers.

4.2.2 Instance availability

The study also investigates the availability of Mastodon instances. In this context, availability simply means whether a server's front page can be loaded successfully. Since Mastodon instances are managed independently,

the study expects that decentralized platforms could have problems with unstable server availability.

The results confirmed this expectation. Many servers maintained relatively stable uptime, but a smaller group of instances experienced frequent server downtime. The results showed that most servers ran without major issues, while roughly one in nine was offline for more than half of the entire study period.

The study also found that large and popular servers were not necessarily more reliable. Even instances with very high activity levels experienced periods of downtime, showing that popularity alone does not guaranty a smooth running operation.

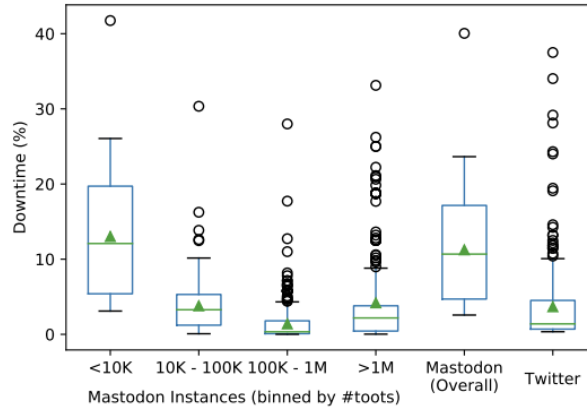


Figure 5: Mastodon instances downtime[19]

When Mastodon was compared to Twitter at a similar development stage, Mastodon performed worse in relation to availability. Twitter was well known for its "Fail Whale" outages during its early years, but the study found that Mastodon's average downtime turned out to be nearly nine times worse than what Twitter had during a similar period, which was around 1.25%.

The study suggests several reasons for these outages. One explanation is that Mastodon is decentralized, which follows that it has this volunteer-based structure where many servers are maintained by individuals or smaller groups who may not always have the resources needed to keep the infrastructure stable over long periods of time. The study also points out that Mastodon still depends on some centralized services. The study also points out that most instances relied on the same certificate provider, which meant that a single problem with that provider could affect many servers at the same time. In addition, many instances were hosted by a small number of large hosting providers, which meant that outages within one provider could impact multiple Mastodon servers simultaneously.

The study also observed that some outages lasted for long periods. Several instances stayed offline for days or even weeks before becoming accessible again.

4.2.3 Federation and user connectivity

The study also investigates how federation affects the structure and reliability of Mastodon as a social network. As we know, federation makes it possible for users on separate instances to follow and interact with users from different instances. This means that Mastodon does not only depend on individual servers, but also on the connections between them. This can create what the authors describe as a "ripple effect", where problems on one instance can indirectly affect other connected instances as well.

The study investigates this further by analyzing Mastodon's follower graph and comparing it with Twitter. The experiment simulated what would happen if "highly connected users" were removed from the network bit by bit. "Highly connected users" refers to users with many followers or followings. In each iteration, the top 1% of these users were removed, after which the study measured how much of the network still stayed connected. The following figure shows the result:

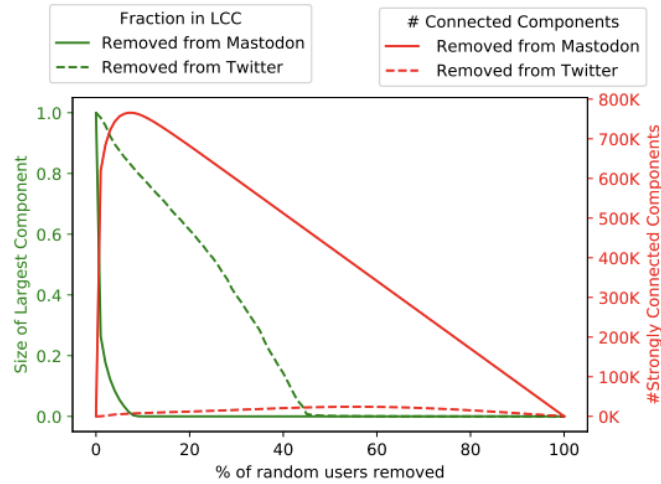


Figure 6: Comparison between Mastodon and Twitter under user removal

The results show that Mastodon reacts much more strongly to the removal of central users than Twitter does. When influential users disappear, Mastodon's network quickly loses its connected structure and becomes fragmented. This means that many users are no longer connected within the same network.

Twitter, on the other hand, remained more stable during the same experiment, even when many important accounts were removed. This suggests that Mastodon’s federated structure is more vulnerable to failures, especially because users may suddenly become unavailable when the instance hosting their account goes offline.

Another experiment the study conducted was to see what would happen if entire instances were removed from the network. Since Mastodon depends on federation, removing the most popular instances can affect large parts of the network. The results showed that the federation graph was more stable than the user graph, meaning that the network didn’t degrade as fast. However, removing large instances still caused significant disconnection across the network.

4.2.4 Replication

The study also goes over what happens to posts when instances are removed from the network, and it explores possible solutions to this problem. The following figure (b) shows the effect of removing instances without using replication. As more connected and larger instances are removed, the availability decreases quickly.

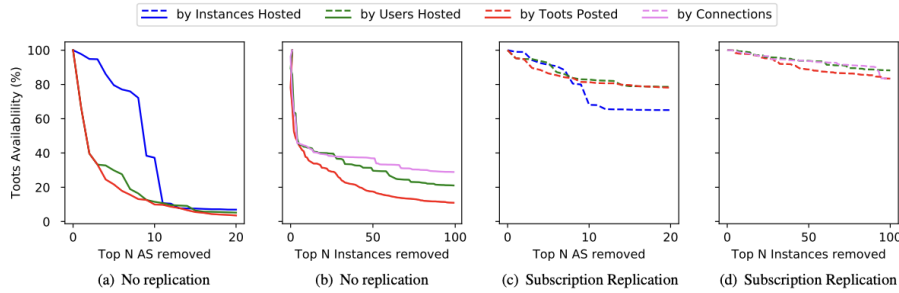


Figure 7: Overview over instances with and without replication

The study then examines what would happen if the data were replicated to other instances. This means that posts would not only exist on the original instance, but also on other connected instances. Figure (d) shows that with replication, posts remain much more available even when important instances are removed from the network.

The way replication works is by storing copies of posts on connected instances. For example, if we are on instance A, and we follow a user on instance B, and the user on instance B makes a post, it’s not only saved on instance B, but also replicated to instance A.

5 Concepts

Before reaching the key part of the ActivityPub protocol and the Fediverse, it is important to become familiar with the key terms used throughout the report and within ActivityPub[22].

5.1 What is an endpoint?

An endpoint is when a specific URL in a web application provides access to its data or functionality. An API allows different systems to send and receive requests, making it possible for them to communicate with each other.

Endpoints are accessed using HTTP methods such as POST and GET, depending on whether the data needs to be sent or received.

In ActivityPub, endpoints represent different parts of the system. For example, the actor, inbox, and outbox are all implemented as endpoints, where each endpoint returns data in a structured JSON format, allowing other servers to retrieve information and interact with the application.

Unlike regular web pages that return HTML rendered in browsers, ActivityPub endpoints always return JSON data.

5.2 What is an object?

In ActivityPub, **objects** are used to represent content (such as blog posts) and entities (such as users and likes) that are shared between systems. For example, a blog post or a user can be represented as an object, while interactions such as likes and follows can also be represented as objects. These objects are used for communication and data exchange between servers in the Fediverse.

Each object has an id and a type. The id is the object's unique global identifier and is typically represented as an HTTPS URL, for example `http://127.0.0.1:8000/ap/post/1` (HTTP is used here for local development and convenience). This also makes it possible to retrieve the object through its URL.

The type describes what kind of object it is. In this project, where we are developing a blog for the Fediverse, blog posts are represented as **Note** objects, which will be discussed in more detail later. Other objects include **Article**, which is used for longer written content, and **Person**, which is used for individual users.

In general, you can say objects are a representation of content or entities within the system. Whether it is an **Article**, **Note**, or **Person** object, an ActivityPub object is defined by its type and a set of properties that describe it. The structure and properties of these objects are defined by the ActivityStreams vocabulary [22], which describes how objects should be represented, published, and consumed by other systems.

All objects are returned in JSON format using the ActivityStreams 2.0 vocabulary. This allows other ActivityPub servers to interpret the data in a consistent way.

5.3 Actor and collections

An **actor** is a central role in the ActivityPub protocol, as it represents an entity that is responsible for performing and receiving different activities within the system.

Activities such as publishing media/content, following other actors, and responding to requests are all carried out by actors. In this sense, the actor can be understood as the main "orchestrator" behind the interactions in ActivityPub.

Actors inherit the general properties of ActivityStreams objects. This means that an actor has an id and a type. The id is the actor's unique identifier and is represented as a URI, while the type describes what kind of actor it is, for example, Person, Group, Service, or Application. For example, a Person actor will usually represent a person, a Group will represent a group of people, while a Service actor can represent a blog.

In addition to these properties, an ActivityPub actor is required to have both an inbox and an outbox.[22] The inbox is used to receive activities from other actors, while the outbox is used to publish activities. The Inbox and outbox are represented as **collections** of activities and are essential for communication between federated systems.

While the inbox and outbox are required, it is also highly recommended that actors provide **followers** and **following** collections. In ActivityPub, a collection is a structured list of objects or actors. The followers collection contains the actors that follow a given actor, while the following collection contains the actors that the given actor follows. These collections make it possible to model the relationships between actors and distribute content across the federated network.

Other collections can also be included. For example, the liked collection contains objects that the actor has liked, meaning it represents the actor's own interactions with content. In contrast, the likes collection is used to represent all like activities belonging to a specific object, showing how many actors have liked the object. Additionally, the shares collection contains Announce activities and is used to represent how often an object has been shared across the network. These collections can help track interactions and engagement across content in a federated environment.

5.4 Inbox and Outbox

In ActivityPub, the inbox and the outbox are central components that enable communication between actors in the federated network. Both are rep-

resented as collections and are used to handle the flow of activities between different actors.

The inbox is used to receive activities from other actors. When another server wants to interact with an actor, such as sending a follow request or delivering content, it sends a request to the actor's inbox. The inbox can be seen as a container for all incoming activities.

The outbox, on the other hand, is used to publish activities created by the actor. These activities can include creating content, such as a blog post, or interacting with other actors, such as liking or following. Other servers can retrieve the outbox to see what activities the actor has performed. The outbox can be seen as a container for all the outgoing activities created by the actor.

Both the inbox and outbox are represented as collections of activities. In ActivityPub, a collection is used to represent a structured list of objects or actors. Collections are used to store data such as followers, following relationships, liked objects, or activities.

ActivityStreams defines two types of collections, **Collection** and **OrderedCollection**. A **Collection** represents a list where the order is not important for example a list of followers, where an ordered list is unnecessary, while an **OrderedCollection** is used when the order of elements matters, such as a list of activities in an outbox where the newest activities are best shown first.

An **OrderedCollection** is a subtype of **Collection**, where activities are stored in an ordered way. Since **OrderedCollection** is a subtype of **Collections**, it will inherit the properties such as `totalItems` and either `items` or `orderedItems`, which are used to store the elements of the collection.

In addition, the inbox and outbox follow specific rules in the ActivityPub protocol. The inbox typically accepts HTTP POST requests, which allow other servers to deliver activities, while the outbox can be accessed through GET requests which will retrieve published activities. The outbox can also accept POST requests when creating new activities from a client. [22]

Overall, the inbox and outbox create the foundation of communication in ActivityPub, as they make it possible to send and receive activities across the federated network, making them an essential part of the protocol.

5.5 Activities

In ActivityPub, the word "activities" is used to describe actions performed by actors[22]. One of the most important activity types is the **Create** activity, which is used to create or publish new objects.

A **Create** activity typically contains an `object` property that contains the actual content being created. For example, when a blog post is published, it is represented as a **Create** activity where the blog post itself is included as an **Note** object.

Another important activity type is **Follow**, which is used when one actor wants to follow another actor. This is especially relevant in the federated system, since it allows actors on different servers to subscribe to each other. A **Follow** activity can later be answered with either **Accept** or **Reject**, which are activities used to indicate whether the follow request is approved or denied.

ActivityPub also supports other activity types, such as **Like**, which is used to react to content, **Update**, which is used to modify an existing object, and **Delete**, which is used to delete an object. While this project mainly focused on **Create** and **Follow**, these examples show that ActivityPub supports a long list of interactions across the federated network.[22]

Activities are generally stored in the actors outbox and can be retrieved by other servers. This allows other actors in the fediverse, such as Mastodon servers, to see what actions have been performed. For example, a Mastodon server can retrieve activities showing that the actor has published a blog post, followed other actors, or liked a specific object.

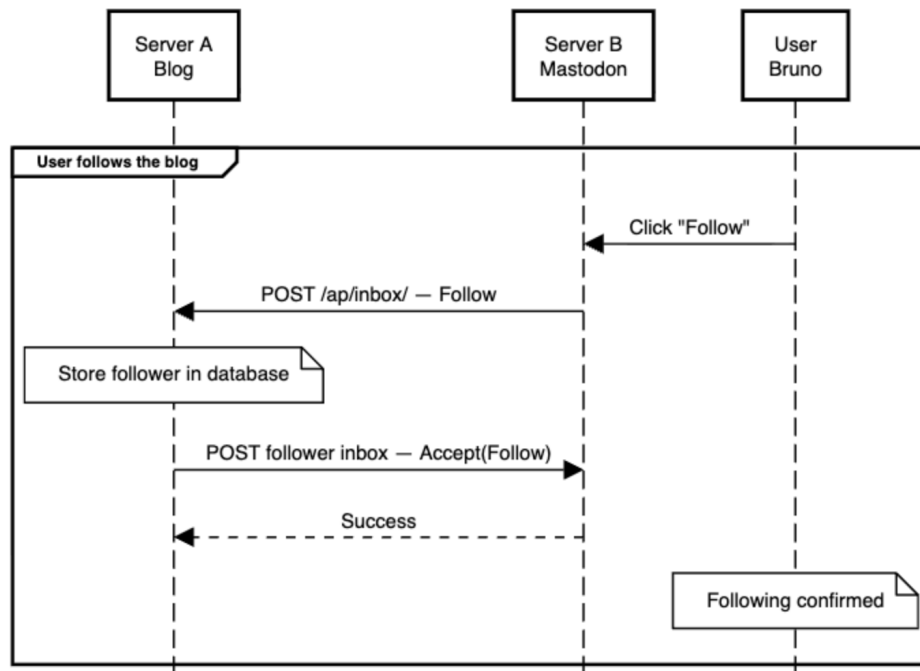


Figure 8: Follow interaction between servers

Figure 8 shows a simplified follow interaction between two servers.

In this example, a user on a Mastodon server, Server B, follows the blog hosted on Server A. When the user clicks "Follow", the Mastodon server sends a **Follow** activity to the blog's inbox endpoint (/ap/inbox).

The blog server receives the request and stores the follower. It then

responds by sending an **Accept** activity back to the follower’s inbox. Once this is completed, we will have a follow relationship is created between the User on Server B and the blog on Server A

Another example of an important interaction in ActivityPub is how content is distributed after a follow relationship, as shown in the following figure:

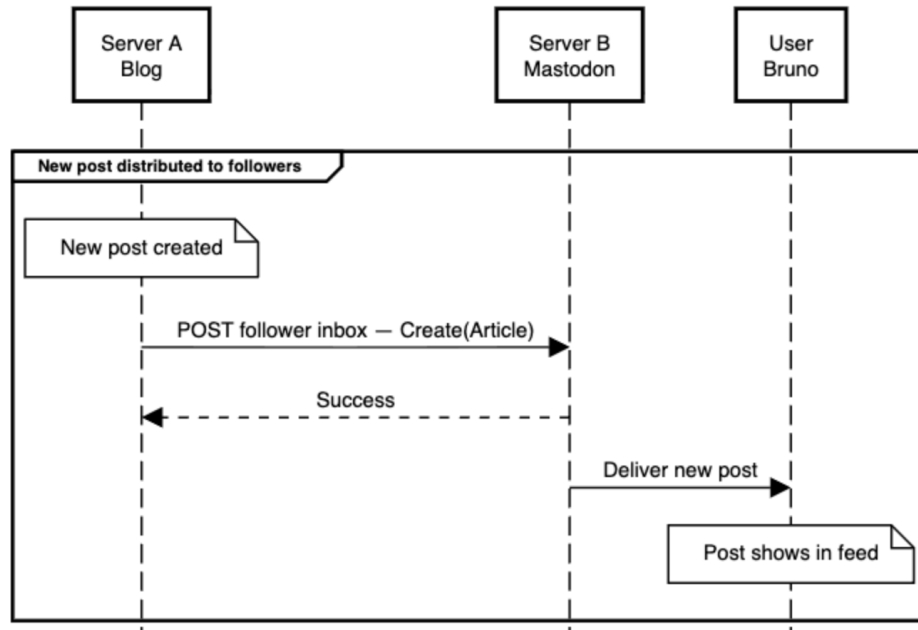


Figure 9: Content distribution example

In this example, the blog on Server A creates a new post. This is represented as a **Create** activity containing an **Article** object, which represents the initial planned blog content, but we later changed the activity to **Note**. The server then sends this activity to the inboxes of its followers, in this case Bruno.

When Server B receives the activity, it processes the content and delivers it to the user. As a result, the new post is shown in the user’s feed.

5.6 WebFinger

The general idea is that an actor can be retrieved directly through a URL that returns data in the ActivityStreams 2.0 format. This allows other servers to fetch information about the actor, such as its inbox, outbox, followers collection, and other properties needed for communication.[16]

However, in the Fediverse, relying on the exact actor URL in advance is not ideal. Since each server can define its own URL structure, there is no consistent way to predict where an actor is located just by the URL. For example, a user `@user1@example.com` might have an actor URL such as

`https://example.com/users/user1`, while another could use a completely different structure like `https://example.com/ap/actor/user1`. This makes it difficult to determine the correct URL without knowing the server's structure.

Instead, WebFinger is used to locate actors by using identifiers such as `username@domain`, which are then translated into the actual ActivityPub actor URL. This allows servers to find and interact with an actor without knowing its exact address beforehand.

For example, when a Mastodon user searches for `@user@domain`, the Mastodon server performs a WebFinger lookup in order to find the corresponding ActivityPub actor resource.

5.6.1 RFC 7033

RFC 7033[21] is the official specification that defines the WebFinger protocol, which is used to discover information about entities on the internet using standard HTTP requests. A key concept in the specification is URIs, which are used to uniquely identify the entity being queried.

WebFinger has several use cases and is not only found in the ActivityPub protocol. For example, RFC 7033 describes examples such as retrieving metadata about a web page, including links to author and copyright information, and discovering an identity provider for OpenID Connect. These examples illustrate that WebFinger is a general discovery protocol and is not limited to ActivityPub.

In ActivityPub, WebFinger is used to translate an account identifier, such as `username@domain`, into the corresponding actor URL. According to RFC 7033, a WebFinger query is performed by sending a GET request to the well-known endpoint `./well-known/webfinger`, which is a standardized endpoint used for resource discovery on a server. The request must include a query component with a `resource` parameter, which specifies the URI of the entity being queried. In other words, it tells the server what information the client is trying to discover. In ActivityPub, the `resource` value uses the `acct` URI scheme, for example, `acct:username@domain`. This tells the server which account or entity the clients want information about.

The server then responds with a JRD (JSON Resource Descriptor), which is a JSON object that describes the queried entity. A JRD can contain several fields, such as `subject`, `aliases`, `properties`, and `links`, where `links` field contains references to related resources through URLs.

The `subject` field identifies the resource that was queried, this is recommended. The `aliases` field can contain alternative URIs that refer to the same entity. The `properties` field can be used to include additional metadata about the entity. Both `aliases` and `properties` are optional. The `links` field is the most important part in this project, even though it is also optional according to the specification. The `links` field is an array where

each element describes a relationship to another resource.

Each object in the `links` array includes fields such as `rel`, `type`, and `href`. The `rel` field describes the type of relationship and is required. Its value must be either a registered relation type or a URI. The `type` field describes the media type of the target resource, for example `application/activity+json`, which tells the client how to interpret the data when accessing the link. The `href` field contains the URL to the related resource.

In ActivityPub, the important link is the one that points to the actor resource. This link is typically identified by having `rel` set to `self` and the media type set to `application/activity+json`. The URL of the actor is provided in the `href` field, which the client can use to retrieve the actor's ActivityPub resource. An example of a response from WebFinger will be:

Listing 1: Example of a WebFinger response taken from our project

```
1 {
2   "subject": "acct:amin@127.0.0.1:8000",
3   "links": [
4     {
5       "rel": "self",
6       "type": "application/activity+json",
7       "href": "https://127.0.0.1:8000/ap/users/amin/"
8     }
9   ]
10 }
11 }
```

This means that another server can query `./well-known/webfinger?resource=acct:username@domain`, extract the `href` value from the response, and then fetch the actor from that URL. In this way, WebFinger links a simple account identifier to the corresponding ActivityPub actor resource. It provides a standardized way to locate an account and retrieve the information needed to interact with it and this is what enables communication across different servers in the Fediverse.

5.7 Security and Keys

5.7.1 HTTP Signature

Once communication happens between two federated servers, there needs to be a way to authenticate the communication and ensure that the message has not been changed or is coming from an unreliable source. This is where HTTP Signatures are used.

HTTP Signatures allow a server to digitally sign an HTTP message. Following the HTTP Signature draft[20], this provides both message integrity and authentication. Authentication means that the receiving server can verify who sent the request, while integrity means that the receiver can verify

that the request was not modified after it was signed.

In ActivityPub, this is very important because the servers send activities directly to each other. For example, when a Mastodon server sends a Follow activity to our inbox, our server should be able to verify that the request was actually sent by the actor in the activity and not by a fake server pretending to be that actor.

The signature is included in the HTTP Signature header, which contains several values such as `keyId`, `headers`, `signature`, and `algorithm`. The `keyId` points to the public key that is used to verify the request. In ActivityPub, this public key is found through the actor endpoint.

The `headers` field describes which HTTP headers were included when generating the signature. The header contains fields such as `(request-target)`, `host`, `date`, and `digest`. These headers are combined into a string in a strict order before the signature is generated. This is important because the receiving server must recreate the exact same signing string in the same order when verifying a request.

The `algorithm` field describes which cryptographic algorithm was used to create the signature. In Mastodon and many Fediverse implementations, the commonly used algorithm is RSA-SHA256.

The `signature` field contains the actual digital signature. It is created by signing the selected HTTP headers with the sender's private key and is base64 encoded. The receiving server uses the public key from `keyId` to verify the signature.

In addition to the HTTP Signature header, ActivityPub implementations use the HTTP Digest header. The `Digest` header contains a SHA-256 hash of the HTTP request body encoded as Base64. This allows the receiving server to verify that the body content has not been modified during the transaction. The `Digest` value referenced inside the HTTP Signature header refers to this HTTP Digest header.

An example of an HTTP Signature header looks like this [20]:

```
Signature: keyId="rsa-key-1",algorithm="hs2019",
  created=1402170695, expires=1402170995,
  headers="(request-target) (created) (expires)
  host date digest content-length",
  signature="Base64(RSA-SHA256(signing string))"
```

Figure 10: Example of an HTTP Signature header (from Signing HTTP Messages)

The interpretation would be:

```
(request-target): post /foo\n
(created): 1402170695
(expires): 1402170995
host: example.org\n
digest: SHA-256=X48E9q0okqqrvdts8n0JRJN30WDUoyWxBf7kbu9DBPE=\n
content-length: 18
```

Figure 11: Example of how an HTTP Signature header is interpreted (from Signing HTTP Messages)

5.7.2 RSA and SH256

For security measures and authentication, many Fediverse platforms, including Mastodon, use RSA, which is a public key cryptographic algorithm. RSA is used with HTTP Signatures to verify that incoming requests were actually sent by the actor they claim to be from.

RSA uses two different keys, a private key and a public key. The private key is kept secret and is used for signing requests, while the public key is shared publicly and is used by other servers to verify the signature.

In ActivityPub, the public key is included in the actor endpoint. This allows other servers to retrieve the key and verify signed HTTP requests, such as Follow activities, inbox deliveries, and other interactions.

Mastodon uses RSA-SHA256 which is a combination of the RSA algorithm and the SHA256 hashing algorithm. SHA256 is used to generate a hash of the requested data, and RSA is then used to sign this hash using the actor's private key. The receiving server can verify the signature using the actor's public key. If the verification succeeds, the request is considered authentic and does not come from a fake source.

6 Implementation and Architecture

The first step in the implementation was to make a simple blog that could function as the foundation for the ActivityPub protocol. Before implementing federation and ActivityPub functionality, we wanted to build a working blogging platform with core features, such as users, posts and profiles.

The blog supports user registration, login and password changes. Each user has their own profile page that displays their posts, together with follower and following counts. Users can create blog posts with a title, body, and hashtags, and other users within the blog can comment on these posts. A search feature was also added later, allowing users to search for other users once ActivityPub functionality had been implemented.

The blog was styled using Bulma, while Django's built in authentication system was used for user management. Pagination was also added to the blog index page, where each page is displayed two posts at a time. Together,

these features created a small usable blogging platform before introducing ActivityPub features.

For page rendering, we created a `base.html` template to define the layout, including the navigation bar, authentication links, search functionality, and the general page structure. This made it possible for other pages such as `detail.html`, which displays individual blog posts, and `index.html`, which displays the blog feed, to extend the base template and only define their own content blocks.

Once the Django blog was completed, we could start to implement the ActivityPub functionality. For this we used the book [22] as our primary reference, as the book had a comprehensive list of steps needed to have a fully functional ActivityPub server running. Since the list of implementation steps was extensive, we had to prioritize the most important features first. This allowed us to focus on implementing the essential ActivityPub functionality required for federation before implementing additional features.

Feature	Description
GET object endpoint	A REST API endpoint that returns a ActivityStreams 2 (AS2) representation of a blog post, allowing other ActicityPub servers to retrieve and show the content
GET actor endpoint	Returns a user as an ActivityPub Person actor with inbox, outbox, followers, preferredUsername, and publicKey
GET collection	Returns a list of objects or activities in AS format, for example followers, inbox or outbox
GET actor collections	Returns collections that are related to the actor, such as inbox, outbox, followers, and following
WebFinger	Allows other servers and clients to discover the actor using an identifier such as name@domain.com
HTTP Signature	Creates a signed Signature header for outgoing ActivityPub POST request using the actors private key, RSA-SHA256
Remote delivery	Delivers Create activities containing Note objects to the inboxes of all followers when a new blog post is published
Incoming Follow	Handles incoming Follow activities, store the follower in the database, and send a signed Accept activity
Incoming Undo	When an Undo(Follow activity is received, the corresponding follower is removed from the database
Incoming Create	Handles incoming Create(Note) activities that contain an inReplyTo field
Outgoing Follow	Allows blog users to search for remote Fediverse actors using WebFinger and send a signed Follow activity to their inbox

Table 1: Overview of implemented ActivityPub features

6.1 Project Structure

The project follows Django’s standard application structure, where we divide the implementation into two applications, **blog** and **users**.

The **users** application is responsible for user management, where we decided to extend Django’s built in **AbstractUser** model with additional fields such as biography, profile picture, and RSA key pair used for HTTP Signature authentication in ActivityPub. The **AbstractUser** is a solution for users since it already provides core features such as usernames, email addresses, passwords, and password validation.

The **blog** application contains the main functionality of the platform. This includes blog posts, comments, templates, forms, and ActivityPub features. It is responsible for both the blogging functionality and the ActivityPub protocol.

The views are separated into different modules. In Django, views are responsible for handling incoming HTTP requests and returning responses such as rendered HTML pages or JSON data.

blog_views.py handles the standard blog functionality, such as creating posts, rendering the blog, and showing post detail pages.

activitypub_view.py implements the ActivityPub endpoints, such as the actor endpoint, outbox, and followers collection. It also handles outgoing federation activities such as actor discovery and sending **Follow** activities.

activitypub_inbox.py handles incoming ActivityPub activities sent to the inbox endpoint. This includes activities such as **Follow**, **Undo** and **Create**.

The application also contains **urls.py**, which works as the routing between URLs and views, and **forms.py**, which handles forms used for blog features, such as creating a post or comments, login, and actor search.

6.2 GET object endpoint

The GET object endpoint ensures that individual blog posts are represented as ActivityStreams 2.0 (AS2) objects. This allows other ActivityPub servers to retrieve and present the content in a federated environment.

Each blog post is represented as an **Note** object. The **Note** type is used for short blogging written content, such as blog posts and provides a way to represent content in a sticky note way.

The Note object contains properties such as **id** (the unique identifier of the post), **type**, **content**, **published**, and **attributedTo** (the actor responsible for the post). Together, these properties make it possible for external servers to interpret the content correctly.

An example of the JSON representation of a **Note** object is shown below:

Listing 2: Example of an AS2 Note object

```
1 {
```

```

2  "@context": "https://www.w3.org/ns/activitystreams",
3  "type": "Note",
4  "id": "https://127.0.0.1:8000/ap/post/1/",
5  "content": "Lorem Ipsum ...",
6  "published": "2026-03-12T11:01:12.483678+00:00",
7  "attributedTo": "https://127.0.0.1:8000/actor",
8  "url": "https://127.0.0.1:8000/post/1/"
9  }

```

The line we will see a few times is `"@context":...`, which is used to interpret the JSON data. It tells the receiver which vocabulary is being used, so that properties such as type, name, and content are understood correctly. In ActivityPub, this is typically set to the AS2 context. This ensures that different systems interpret the data in the same way.

6.3 GET actor endpoint

The next step in the ActivityPub implementation is to create the GET Actor endpoint. GET actor endpoint is basically used to represent the blog as an ActivityPub actor

In this project, each local user is represented as a **Person** actor in the blog. A **Person** actor represents an individual user in the ActivityPub protocol and is used to publish posts, receive activities, and interact with other federated actors. In contrast, an **Application** actor would typically represent a software application, and **Service** would represent an automated service.

The actor is available through the `/ap/users/username/` endpoint, which returns a JSON response that is represented using the ActivityStreams 2.0 vocabulary.

The GET actor endpoint has all required actor properties such as `id`, `type`, `inbox`, and `outbox`.^[22] The `id` uniquely identifies the actor, while the `inbox` and `outbox` define how the actor receives and publishes activities. In addition, the endpoint includes a `preferredUsername`, which can later be used to support WebFinger later.

In addition to the required properties, the actor endpoint also exposes collections such as `followers`, `following` and `liked`. These collections describe relationships and interactions between the different actors in the federated network.

The endpoint returns data in the ActivityStreams JSON format (application/activity+json), which allows other ActivityPub servers to interpret the actor in a consistent way. This makes it possible for external servers to discover the actor and interact with it through mechanisms such as following and inbox delivery, which will be implemented later.

The actor endpoint defines how the blog is exposed to the federated network and enables other servers to interact with it in a standardized way.

6.4 GET collections

After implementing the actor endpoint, the next step is to implement collections.

In the implementation, collections are exposed through endpoints such as `/ap/users/{username}/outbox/`, `/ap/users/{username}/inbox/`, and `/ap/users/{username}/followers/`. These endpoints return data in ActivityStreams 2.0 format, allowing other ActivityPub servers to retrieve a information from the server.

As mentioned earlier, collections can be represented as either `Collection` or `OrderedCollection`. In this implementation, the collections are represented as `OrderedCollection`, since this allows the elements to be returned in a defined order using the `orderedItems` property. This is mostly relevant and important for the outbox, where posts are returned with the newest items first.

Each collection includes properties such as `id`, `type`, and `totalItems`, along with `orderedItems`, which contains the actual elements of the collection. The `totalItems` property indicates how many elements the collection contains, while `orderedItems` contains the list itself.

For example, the outbox endpoint returns an `OrderedCollection` where `totalItems` indicates how many blog posts exist in total, while `orderedItems` contains `Create` activities. Each `Create` activity represents the action of publishing a blog post. The blog post itself is included inside the activity as an `Note` object using the `object` property. This makes it clear both what the content is and that it was created by the actor. The order of the list is presented with the newest posts first.

Each `Create` activity also includes fields such as `to` and `cc`, which describe who the post is shared with. The `to` field is used for the main audience, in this case the public, while `cc` is used to include additional recipients, such as the followers of the actor.

The followers endpoint is also returned as an `OrderedCollection`. In this case, `totalItems` indicates how many remote actors follow the user, while `orderedItems` contains the actor URLs of the followers. Each follower is represented by its actor URL, which allows other servers to identify and interact with these actors across the federated network.

6.5 Follower model

Before implementing the inbox and handling Follow activities, a data model was needed to persist information about remote actors who follow local users.

The model stores information about each remote follower. It includes a reference to the local `user` being followed, the `actor_url`, which uniquely identifies the remote ActivityPub actor, the `inbox_url`, which is used to

deliver activities back to the follower, and the `display_name`, which is a more readable name retrieved from the `preferredUsername` field. The model also stores a `follow_on` timestamp to indicate when the follow relationship was created.

To prevent duplicate follow relationships, we use a unique constraint on the combination of `actor_url` and `user`. This ensures that the same remote actor can follow a specific user in our blog, while allowing the same remote actor to follow multiple users on the platform.

6.6 WebFinger

In federated systems such as the Fediverse, users and services are not identified only by local usernames, but globally unique identifiers that include a domain. In ActivityPub, actors are typically identified by URLs. This means that the domain is an important part of the actor's identity, as it says which server is responsible for the actor and where the actor's data can be retrieved.

Users do not usually interact with full URLs directly, but a more human-readable format: `acct:username@domain`. This format resembles an email address and is easier to use when searching for users across different servers. To translate it into a usable actor URL, the webfinger protocol is used.

WebFinger is a discovery mechanism that allows clients and servers to retrieve information about a resource identified by a URL. WebFinger maps an `acct:` identifier to the corresponding actor URL.

6.6.1 How WebFinger works?

In our implementation WebFinger is an endpoint that processes incoming requests containing a resource parameter. This parameter typically contains a value such as `acct:username@domain`. The purpose of the endpoint is to translate this identifier into the corresponding Activitypub actor URL.

It begins by validating that the `resource` parameter is in the request. If the parameter is missing the server returns a error response. The value is then parsed by removing the prefix: `acct:` and splitting the remaining string into the username and the domain.

The username is used to identify a local user in the database, and the domain is used to verify that the request is intended for the current server. This is done by comparing the extracted domain with the value returned by `request.get_host()`, which is the current host. If the domain and the value do not match, then the request is rejected. This ensures us that the server only responds to WebFinger requests for the actors that its actually responsible for.

If both username and domain are valid, and the user exists in the database, then the server will return a JSON response in the JRD format.

WebFinger helps external servers and clients discover the correct actor URL from a simple identifier. It allows actors to be located and interacted with across different domains.

6.7 Deployment

After WebFinger, we wanted to test our implementation in practice. We had developed a Django based blog together with enough necessary ActivityPub components, such as actors and endpoints. In theory, our server should allow other servers, such as Mastodon, to find and interact with our actors. This allowed us to test which parts of our implementation were working correctly and which were not.

For deploying the Django blog, we decided to use the free version of **Render.com**, as it gave an easy way to host our application over HTTPS. Since ActivityPub requires publicly accessible endpoints for communication, it made sense.

Render allows deployment directly from a Git repository, where the application/blog is automatically built and started based on the specified configuration settings, such as the version of Python and root directory and build commands. Once **Render** deploys, it will install the required dependencies from `requirements.txt`, runs database migrations, and start the application using **Gunicorn**, which is a server using WSGI protocol.

For the database, we initially used **SQLite**, which is Django's default database. **SQLite** is okay for development and testing purposes because it's easy to set up. However, later in the implementation process, we decided to use **PostgreSQL**, which is more suitable for production environments and persistent data. During deployment, we experienced issues with data persistence when using **SQLite**, especially after redeployments.

Deploying the application through **Render** also ensures that the application is assigned a public URL. In our case, this is `https://activitypub-blog.onrender.com/`, which makes it accessible to other servers in the Fediverse.

6.8 Public Key

After implementing WebFinger and actor discovery, deploying the blog, and confirming that Mastodon could discover our actor, the next step was to implement public key cryptography as a security mechanism. Most Fediverse servers require cryptographic verification before allowing interactions such as follows, likes, inbox delivery, and content federation.

Mastodon uses HTTP Signatures with RSA-SHA256 for authentication and verification of incoming requests. Without this mechanism, Mastodon users would not be able to follow our actors.

Our implementation starts with generating an RSA private key for each user. The private key is generated with a key size of 2048 bits and a pub-

lic exponent of 65537, which is the standard configuration used for RSA cryptography.

Once the private key has been generated, a corresponding public key will also be generated from it. Both keys are then serialized into the PEM format before being stored in the database. The private key is secret and is later used when signing outgoing HTTP requests. In contrast, the public key is exposed through the actor endpoint, which allows ActivityPub servers to retrieve it and verify signed requests from our server.

To avoid generating new keys every time, the implementation only generates a key pair if the user does not already have a public key to his actor. This ensures that each actor maintains a consistent identity across the Fediverse and the interactions.

The public key is part of the actor representation using the `publicKey` property:

Listing 3: Public key included in the actor endpoint

```
1 "publicKey": {
2   "id":
3     "https://activitypub-blog.onrender.com/ap/users/
4     amin/#main-key",
5   "type": "Key",
6   "owner":
7     "https://activitypub-blog.onrender.com/ap/users/
8     amin",
9   "publicKeyPem": "-----BEGIN PUBLIC
10     KEY-----\...\n-----END PUBLIC KEY-----\n"
```

6.9 HTTP Signature

After implementing the public/private key part, we started working on HTTP Signatures. For this implementation, we used Mastodon’s documentation[15] as a reference for how an HTTP Signature header should be structured, since one of the main challenges was understanding the format required by the federated servers.

Our implementation signs outgoing ActivityPub POST requests such as Follow, and Create activities using the RSA-SHA256 algorithm.

First, the ActivityPub body is converted into JSON and encoded as UTF-8, because the request body must be converted into bytes before it can be hashed and signed. A SHA-256 digest is then created from the body and Base64 encoded. The digest is Base64 encoded because HTTP headers can only contain text values and not binary data. This digest is included in the HTTP **Digest** header so that the receiving server can verify that the request body has not been changed.

We then construct a string for the HTTP header, which includes the values (`request-target`), `host`, `date`, and `digest`. These are in strict

order because the fediverse servers recreate this in exact same string when they signature verify.

The **Date** header is included together with the **Host** header, which identifies the target server. The **request-target** field contains the HTTP method and request path in lowercase format.

We then load the user's private RSA key from the database and use it to generate the signature using SHA-256 hashing together with PKCS1 v1.5 padding. The generated signature is then Base64 encoded before being included in the final HTTP Signature header.

The final Signature header contains the **keyId**, which points to the actor's public key, **headers**, which are the headers included in the signature, **signature**, the base64 encoded signature, and the algorithm used, **rsa-sha256**.

When a remote ActivityPub server receives the request, it uses the **keyId** field to retrieve the sender's public key from the actor endpoint. The server can then recreate the signing string and verify the signature using the public key. If the verification succeeds, the server can confirm that the request was sent by the correct actor and that the request content has not been modified.

The HTTP Signature implementation is exposed through the **signed.post()** helper function, which automatically creates the required headers and sends signed ActivityPub requests to remote inboxes, such as Mastodon.

6.10 Inbox and Follow

The inbox endpoint is responsible for receiving incoming ActivityPub activities from other servers. In this implementation, each local user has a dedicated inbox endpoint exposed through the route `/ap/users/<username>/inbox/`. As a result follow requests and other activities can be handled in relation to a specific local actor.

The inbox accepts HTTP POST requests[22] containing ActivityPub activities in JSON format. When a request is received, the server parses the JSON body and checks the activity type.

The primary activity handled in this project is the **Follow** activity. When another ActivityPub server wants to follow a local actor, it sends a **Follow** activity to the actor's inbox. The implementation then verifies that the follow request is actually intended for the local actor by comparing the **object** field with the actor's own URL.

If the request is valid, the server retrieves the remote actor resource and extracts the actor's inbox URL. This inbox URL is then stored in the local database together with the actor URL and display name using the **Follower** model mentioned earlier.

After storing the follower, the server creates an **Accept** activity and delivers it by sending an HTTP POST request directly to the follower's inbox URL.

The inbox also supports a simple unfollow mechanism through the **Undo** activity. If an **Undo** activity containing a previous **Follow** activity is received, the follower is removed from the database.

A sequence diagram illustrating the complete federation flow is presented later.

6.11 Content Delivery to Followers

One of the most important features of ActivityPub is the ability to deliver content between federated servers. When a user publishes a blog post, the server must actively push the content to the inboxes of all followers.

According to the ActivityPub book [22] additional features such as delivery queues, retry strategies, asynchronous processing, and failure handling are needed to ensure reliable federation between servers. Our implementation focuses on the core federation functionality and does not currently support features such as queueing or retry mechanisms.

In our implementation, we send ActivityPub **Create** activities to the inboxes of followers whenever a new blog post is published. When a user creates a new post through the Django application, the server first stores the post in the local database. After the post has been saved, the function `deliver_post()` is called. This function is responsible for constructing and distributing a federated ActivityPub message.

The message is represented as a **Create** activity that contains a **Note** object. Activities are used to describe actions performed by actors, while objects represent the actual content. In this implementation, the **Create** activity represents the publication of a new blog post, while the **Note** object contains the actual post data.

The activity includes two addressing fields. The `to` field is set to the ActivityStreams Public address, indicating that the content is publicly visible. The `cc` field references the author's followers collection. This is the same convention used by Mastodon when distributing public posts.

After the activity has been constructed, the server retrieves all followers related to the actor from the database. Each follower stores an inbox URL, which represents the endpoint where the remote server receives incoming ActivityPub activities.

The implementation then iterates through all followers and sends the **Create** activity to each inbox using an HTTP POST request with the content type `application/activity+json`. In this way, the server actively distributes newly published content instead of relying on remote servers to continuously poll for updates.

The delivered activity contains metadata about the action, as well as the embedded **Note** object containing the blog post itself. This allows remote servers not only to retrieve the content, but also to understand which action

was performed on it. In this case, the receiving server can interpret the activity as the publication of a new public note by the actor.

Through this implementation, remote ActivityPub platforms, such as Mastodon, can receive and process newly published blog posts. Once a remote server receives the **Create** activity, it can store the object locally and distribute the content to users following the actor. As a result, posts published through the Django application can appear directly in feeds across federated servers.

6.12 Incoming Undo

In addition to **Follow**, the inbox also handles basic **Undo** activities for unfollow requests as mentioned earlier. Unfollowing an actor is performed by sending an **Undo** activity that contains the previous **Follow** activity as its object.

When the server receives an **Undo** activity, the implementation first retrieves the object and checks whether it has the type **Follow**. This confirms that the follow was performed. If this is the case, the actor URL from the activity is used to identify the follower in the database. The corresponding follower entry is then removed from the **Follower** model.

This allows other ActivityPub platforms, such as Mastodon, to unfollow actors on our platform.

6.13 Incoming Create

Our implementation also supports incoming **Create** activities, which are used to handle replies from other ActivityPub platforms.

When the inbox receives a **Create** activity, the JSON body is parsed and the object is retrieved from the activity using the **object** property. The implementation so far only accepts objects of type **Note**, since Mastodon and similar Fediverse platforms use **Note** objects for replies.

After retrieving the object, the implementation checks whether an **inReplyTo** field is present. This field identifies which ActivityPub object the reply belongs to. If the field is missing, the activity is ignored, as the implementation only focuses on replies to local blog posts.

If the **inReplyTo** field exists, the implementation finds the corresponding post by matching the referenced ActivityPub ID with a post stored in the database. If no matching post exists, the server returns an error response.

When the referenced post is found, a new entry is created and we store the reply content together with the remote actor URL in the database. The **actor** field from the activity is used as the author identifier, while the **content** field from the **Note** is stored as the body of the comment. This allows incoming replies to be stored and displayed as comments on the corresponding blog post.

This allows users on remote ActivityPub platforms to reply directly to blog posts, and their replies will appear as comments in our actors blog posts through federation.

The `inReplyTo` property is used to indicate that an object is a reply to another object. In this project, it is used to connect incoming replies to local blog posts.

6.14 Outgoing Follow

The outgoing follow functionality allows users on our platform to follow actors across the Fediverse. The implementation provides a search feature where users can search for remote actors using the format `@user@domain` or `user@domain`. This is necessary because ActivityPub actors can use different URL structures depending on the platform, meaning that the exact actor URL cannot be known in advance. To solve this problem, the implementation uses the WebFinger protocol to discover the actor's identifier and the corresponding ActivityPub actor URL.

The implementation begins by splitting the user handle into a username and domain. A WebFinger request is then sent to the remote server using the endpoint `/.well-known/webfinger?resource=acct:user@domain`. The returned JRD response contains a `self` link pointing to the actor resource in ActivityStreams format. The implementation extracts this actor URL and sends another request to retrieve the full actor object.

Once the actor object has been retrieved, the implementation obtains the remote inbox URL from the actor object. The inbox URL is required to deliver ActivityPub activities to the remote actor. The implementation then stores the remote actor information in the database before generating a signed `Follow` activity.

The `Follow` activity is signed using HTTP Signatures together with the actors RSA private key and delivered to the remote inbox using an HTTP POST request. This allows users on our platform to follow actors on other Fediverse platforms that implement the ActivityPub protocol.

The remote actor information is stored in the `Following` model, which follows the same structure as the `Follower` model but represents actors that our blog users follow.

7 Architecture

7.1 Sequence diagram

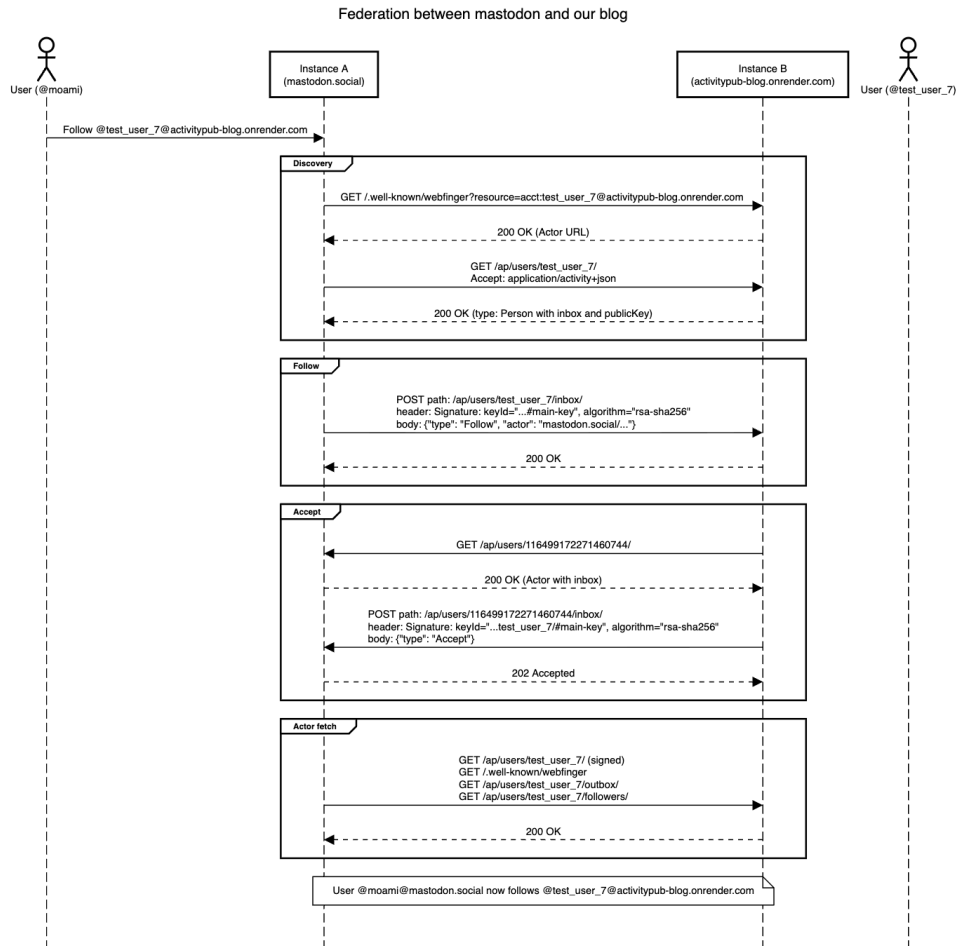


Figure 12: Extended sequence diagram of Follow and Accept activities

This figure shows that federation in ActivityPub is not handled with a single request, but instead through a sequence of steps between federated servers.

One important detail shown in the sequence diagram is that the follow interaction depends on discovery before the actual **Follow** activity can be delivered. Mastodon is not able to send a follow request from only the identifier `@test_user_7@activitypub-blog.onrender.com`. It first does a WebFinger lookup to translate the handle into the corresponding ActivityPub actor URL. For example, `@test_user_7` from our blog would have the URL, `https://activitypub-blog.onrender.com/ap/users/test_user_7/`, while our Mastodon.social user, `@moami@mastodon.social` would have the URL,

<https://mastodon.social/ap/users/116499172271460744>. This shows that different ActivityPub servers can use different URL structure for their actors.

Mastodon then retrieves information about the remote actor, including the inbox endpoint and public key.

After the actor has been discovered, Mastodon sends a signed **Follow** activity to the inbox endpoint of the blog actor. The activity is delivered as an HTTP POST request including both the JSON body and the HTTP signature header.

Another important step shown is that our blog fetches information about the Mastodon actor after the **Follow** activity has been received. This is important because the blog needs to get the inbox endpoint of the actor before an **Accept** activity can be sent back to Mastodon.

Before the **Accept** activity is sent, the request needs to be signed using the actor's RSA private key. Mastodon can then verify the signature using the public key shown through the actor endpoint.

Although the follow request has been accepted, the federation still continues. Mastodon will do extra request to retrieve collections such as the followers collection and outbox. This shows that federation is not just a single interaction, but more of a synchronization process between servers.

In general, federation interactions in ActivityPub follow a similar structure, with actor discovery, endpoint retrieval, activity delivery, and verification between federated servers.

7.2 C4 model

To illustrate the architecture of the system, we use the C4 model since it describes the software system in different levels. In this project, we use the first three levels of the model, Context, Container, and Component. The fourth level, code, is not included.

7.2.1 Context

The context diagram shows the system from a high level perspective. It shows how users interact with the blogging platform and how the platform communicates with other ActivityPub servers such as `mastodon.social`.



Figure 13: C4 model, Level 1: Context

7.2.2 Containers

The container diagram shows the main containers of the system. It shows how the browser, Django application, PostgreSQL and other Fediverse servers interact with each other.

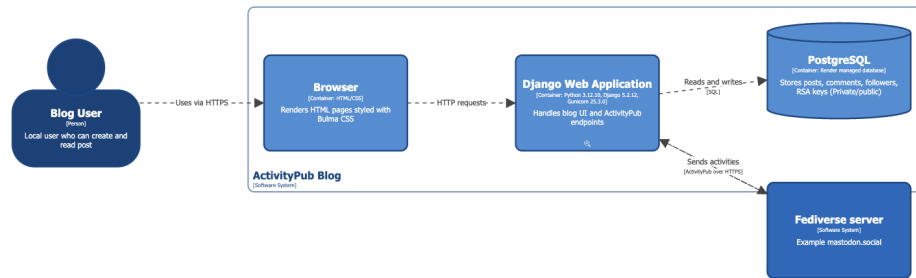


Figure 14: C4 model, Level 2: Containers

7.2.3 Components

The component diagram shows the structure of the Django application. It illustrates how the blogging functionality, ActivityPub endpoints, inbox handling, WebFinger implementation, HTTP Signature generation, and database models are separated into modules and how they interact.

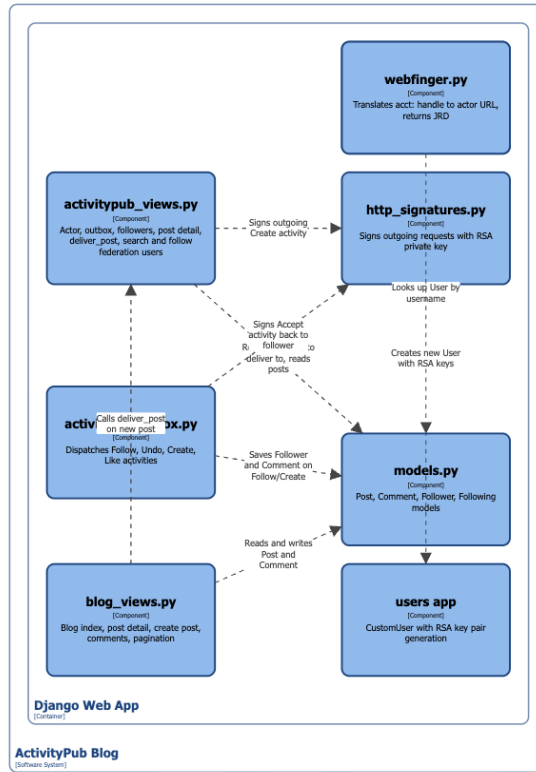


Figure 15: C4 model, Level 3: Components

8 Testing

For testing our implementation and functionality, we used a combination of print statements, reading JSON outputs to verify that they matched the ActivityPub W3C standards, and using `Mastodon.social` as the main Mastodon instance to confirm that the implemented functionality worked correctly in practice.

We have chosen not to include all performed tests in this report. A large part of the testing consisted of inspecting JSON responses from endpoints such as WebFinger, actor, inbox, outbox, and collections, and comparing them with examples from the ActivityPub book[22], and Mastodon documentation[13]. Including all the testing and JSON responses would make the testing section unnecessarily long. Instead, we selected a few examples to show the idea behind the testing.

8.1 WebFinger URL Testing

Testing WebFinger was split into two parts, we wanted to verify that JSON outputs matched the expected WebFinger format. Second, we wanted to

confirm that external Fediverse platforms could find users from our platforms, since WebFinger is the main reason that other fediverse instances can find us.

The initial testing was done locally by directly accessing the WebFinger endpoint through the URL:

`http://127.0.0.1:8000/.well-known/webfinger?resource=acct:amin@127.0.0.1:8000`

The returned output was:

Listing 4: WebFinger output for amin on local server 127.0.0.1:8000 for request

```
1 {  
2   "subject": "acct:amin@127.0.0.1:8000",  
3   "links": [  
4     {  
5       "rel": "self",  
6       "type": "application/activity+json",  
7       "href": "https://127.0.0.1:8000/ap/users/amin/"  
8     }  
9   ]  
10 }
```

The output confirmed that the WebFinger endpoint correctly returned a `subject` together with `self` link that points to the corresponding ActivityPub actor endpoint.

8.2 WebFinger Mastodon Testing

After verifying that the local WebFinger endpoint returned the expected JSON structure, the next step was testing whether external Fediverse platforms could discover our users through federation.

For the initial federation testing after deployment, we used `https://mastodon.social/` and tried to find our actor. We had created a user named `amin` with two posts created. We then searched for the account using its handle (also called WebFinger identifier), `@amin@activitypub-blog.onrender.com`. The result was:

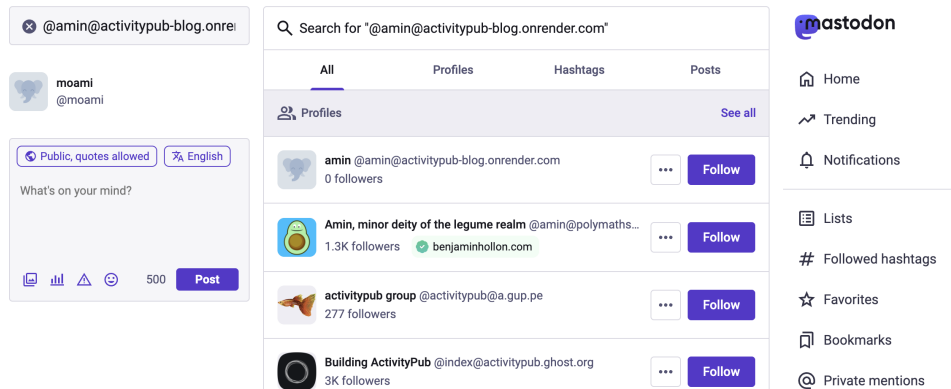


Figure 16: Searching @amin@activitypub-blog.onrender.com

If we click the first result, which corresponds to our actor, we get the following profile:

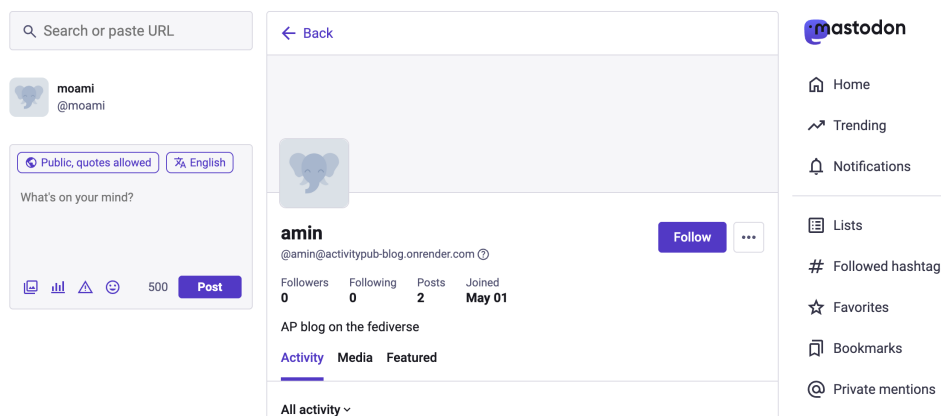


Figure 17: Profile of @amin@activitypub-blog.onrender.com

From this, we can confirm that our WebFinger implementation works as intended. The actor `amin` can be found successfully, and we can see that the number of posts the actor has created matches with the two we created.

8.2.1 HTTP Signature testing

To test whether we got the right HTTP Signature was a combination of re-deploying and testing the functionality and using print statements to debug and print the HTTP Signature header to see if it matched Mastodon's expected format. Our end result from the print statements was the following:

Listing 5: HTTP Signature header for request

```
1 {
2   'Host': 'mastodon.social',
```

```

3  'Date': 'Sat, 16 May 2026 13:15:56 GMT',
4  'Digest': 'sha-256=EfzTm05UhmbScWin3Nn024...',
5  'Signature':
6      'keyId="https://activitypub-blog.onrender.com/...',
7      'headers="(request-target) host date',
8      'digest"',
9      'signature="cnxZYHADuEOVQs6Qj..."',
10     'algorithm="rsa-sha256"',
11 'Content-Type': 'application/activity+json'
12 }

```

As we can see, the **Signature** header contains the four fields, **keyId**, **headers**, **signature**, and **algorithm**. Mastodon.social will respond with a HTTP 202 and then with a 200 to confirm that the signature has been successfully verified.

9 Discussion

The implementation of this project also showed that decentralization introduces some complexity compared to a centralized system. While standard web things such as HTTP and JSON are used, federation requires multiple mechanisms to work together in order to function. Features such as WebFinger, actor endpoints, inbox and outbox, collections, public keys, HTTP signatures and activity delivery must all work together in order to communicate with other fediverse platforms such as Mastodon.

9.1 Django implementation

Django was chosen as the web framework for this project, because it already handled standard things of a web application for us. Things like URL routing, authentication, database management, and handling HTTP requests and responses worked great for a blog, which meant we could focus more on the ActivityPub specific parts. Since ActivityPub communication is through HTTP endpoints returning JSON data, Django's view system made it very straightforward to expose things like actor, inbox, outbox, and WebFinger endpoints.

Our blog initially started as a traditional blog where users could create posts and interact with them locally. After adding ActivityPub functionality, we used Django models to store posts, followers and keys, while ActivityStreams objects and activities are generated with Django views. This made it possible to represent posts as federated ActivityPub objects and deliver them to remote servers such as Mastodon.

What Django did not help with was the federation layer, where all the ActivityPub functionality had to be implemented from scratch. In practice, this meant that a mistake in something as small as an incorrectly formatted

HTTP header or a missing JSON field could break communication with Mastodon.

9.2 Challenges during implementation

One thing we did not expect was how much difference there was between what the ActivityPub specification says and what Mastodon actually expects. ActivityPub allows several ways to do things, but Mastodon has its own preferences.

An example of this was the choice between the **Article** and **Note** object types. At the start blog posts were represented using the **Article** type, since it matched the idea of longer form written blog content. However, during testing with Mastodon, **Article** objects were primarily shown as clickable links showing only the title of the post. Opposite to **Article**, **Note** objects were rendered directly in the Mastodon feed similarly to native posts on the platform. Because of this, the implementation was later changed to use **Note** objects instead, as this provided a more natural presentation and improved communication and compatibility with existing Fediverse platforms. This illustrates how compatibility in the Fediverse is not only affected by the specification itself, but also how individual platforms are designed and implemented in practice.

It's also worth mentioning that working and testing with Mastodon became more important than book specifications, as some implementation decisions, such as using **Note** instead of **Article**, were influenced by how Mastodon renders and processes activities.

Debugging was another practical difficulty. When something did not work, Mastodon often provided limited information when federation failed. For example, the follow request would just fail without us knowing. This made it hard to know whether the problem was in our JSON structure, our HTTP signature, or something else. Much of the debugging process involved sending requests, checking our server logs on Render with print statements, and comparing our output with what Mastodon documentation described as the expected format.

Testing federation locally was not possible since Mastodon requires publicly accessible HTTPS endpoints. This meant that every change had to be deployed before it could be tested against a real Mastodon instance. As mentioned earlier, we used a free version of Render for deployment, but free hosting also had its limitations. For example, the server would enter sleep mode after inactivity. Furthermore, each redeployment of the server took several minutes to complete, which slowed down both development and debugging.

In contrast, testing the Django part of the system was more straightforward. Since the application could run locally during development, changes

were immediately reflected in the local environment, and Django provided clear runtime errors and debugging information whenever there was an issue.

9.2.1 Security mechanisms

One of the challenges faced during the implementation of the ActivityPub protocol was to figure out how security and authentication worked in practice, and how these mechanisms should be implemented correctly. In our implementation, we used RSA key pairs together with HTTP Signature to authenticate outgoing ActivityPub requests such as Follow, and Create activities.

Another challenge regarding security, was understanding how the HTTP Signatures should be structured in order to communicate correctly with Fediverse platforms such as Mastodon, especially how to test whether the generated HTTP Signature headers matched what is expected by the Fediverse and Mastodon.

Much of our testing process was inspecting generated headers and comparing them with Mastodon documentation, and repeatedly redeploying and testing follow request between Mastodon.social and our blog.

For example, we later found out that the date format in the HTTP header should follow the format `Date: Mon, 22 Oct 2013 14:36:43 GMT`, while on Mastodon documentation they excluded the weekday, which appeared to be assumed. Small formatting differences such as this could make the signed request fail.

But once a user on Mastodon.social successfully followed a user on our blog, we could confirm that the HTTP Signature implementation was working as intended.

Another challenge was to understand the cryptographic part behind the authentication process, especially how RSA key pairs should be generated for each user on the platform.

9.3 Limitation and future work

There are several limitations in our current implemented ActivityPub protocol and blogging server. For example, since we are using the free version of Render, the server will experience downtime if it is inactive for a longer period of time, and it also has a total of 750 hour up time each month.

Other examples would be, activities such as delete or full unfollow support, full thread support, and content sharing are not implemented. However, we are more interested in discussing the most important limitations, especially those related to security, production ready deployment, and improvement of our current implementation.

From our perspective, improving the security and scalability of the existing implementation is more important than just implementing additional ActivityPub functionality.

9.3.1 HTTP Signature

One limitation of our implementation is that HTTP Signature verification is performed only in one direction. We sign outgoing messages, but we do not validate incoming signatures. This means that when remote Fediverse platforms send activities such as Follow or Create to our inbox, our server accepts the requests without verifying the HTTP Signature or validating the sender's identity. In theory, this means that a malicious server could send fake activities to our server and pretend to be another actor that it claims to be.

Implementing incoming signature verification would complete the HTTP Signature implementation, which currently only supports outgoing requests. We would start with fetching the sender's public key from their actor endpoint, then reconstruct the signing string from the incoming request header, and comparing the result against the provided HTTP Signature. Implementing this part would be one of the main priorities for future improvements.

9.3.2 Scalability

Our current implementation can have some scalability limitations. For example, when another server requests an actor's outbox, our implementation currently returns all activities at once. This is acceptable for the current prototype, since most actors only have either no or small activities in their outbox. However, if the number of the posts and activities grows, this approach could become inefficient and result in unnecessarily large responses that would take more time to process.

The outbox is currently represented as an `OrderedCollection` that contains all activities directly in the `orderedItems` field. A more scalable solution would be to use `OrderedCollectionPage`, where the outbox is split into smaller pages instead of returning everything at once.

Another scalability limitation in our current implementation is how posts are delivered to followers. The problem is, when a user on our blog creates a post, the server immediately sends the corresponding `Create` activity to each follower's inbox. This again works for a small prototype, but it does not scale well when the number of followers increases or when remote servers are unavailable.

A more scalable solution would be to implement asynchronous delivery and retry mechanisms. This would ensure that in case of deliver failures, such as when a server is offline, we would still retry the delivery. In addition,

using a queue would make it easier to handle situations where a large number of followers need to receive the same activity.

Another delivery optimization would be to implement shared inboxes. Currently, activities are sent to each follower inbox. If many followers belong to the same remote instance, the same activity will be sent multiple times to the same server, which is inefficient. With shared inboxes, the activity only needs to be delivered once to the remote instance, which can then distribute it internally to its users.

9.4 Implementation of Follower model

Initially, our `Follower` model used a unique `actor_url` field, which was unique for the entire database:

```
1 actor_url = models.URLField(unique=True)
```

This design meant that a remote ActivityPub actor could exist only once in the follower table, which means that only one user on our blog could be followed by a specific remote actor from a different ActivityPub server.

The problem was discovered during federation testing with Mastodon, where our blog initially looked like it was working correctly. However, after creating a second blog user and attempting to follow that user from the same Mastodon account, the follow relationship only remained active for around 20 seconds before disappearing again. Mastodon still appeared to send the follow requests, but our follower count was reset from 1 back to 0, indicating that the follow relationship was not being stored correctly on our servers.

To solve this problem, we removed the `unique=True` constraint, and instead made the combination of `actor_url` and `user` unique:

```
1 class Meta:
2     constraints = [
3         models.UniqueConstraint(
4             fields=["actor_url", "user"],
5             name="unique_follower"
6         )
7     ]
```

This implementation allows the same remote ActivityPub actor to follow multiple users on our blogging platform while still preventing duplicates for the same user.

10 Conclusion

This project demonstrates how a Django blogging platform together with the ActivityPub protocol can be implemented to participate in the Fediverse. A functional blogging platform was first developed and then extended with core ActivityPub features, including WebFinger, actor endpoints, inbox and

outbox handling, HTTP Signatures, follower management, and content delivery.

Testing with **Mastodon.social** confirmed that the implementation works as intended. Features such as actor discovery, incoming and outgoing follow activities, content delivery to followers, and incoming replies to blog posts from remote actors were supported.

The project also demonstrates how the core components of ActivityPub work together to enable federation between independent servers.

Although several improvements could be made, such as incoming signature verification, scalability optimizations, and support for additional ActivityPub features, the implementation provides a functional foundation for a federated blogging platform based on the ActivityPub protocol.

References

- [1] Activity streams 2.0. <https://www.w3.org/TR/activitystreams-core/>.
- [2] Activitypub w3 documentation. <https://www.w3.org/TR/activitypub/>.
- [3] Bulma Documentation. <https://bulma.io/documentation/>.
- [4] Cryptography rsa. <https://cryptography.io/en/latest/hazmat/primitives/asymmetric/rsa/>.
- [5] Cryptography rsa-256. <https://infosecwriteups.com/digital-signature-using-sha-256-with-rsa-encryption-0cfd9cbea043>.
- [6] Django auth. <https://docs.djangoproject.com/en/6.0/ref/contrib/auth/>.
- [7] Django build a blog. <https://realpython.com/build-a-blog-from-scratch-django/>.
- [8] Django custom user model. <https://learndjango.com/tutorials/django-userprofile-model>.
- [9] Django tutorials on login/logout. <https://learndjango.com/tutorials/django-login-and-logout-tutorial>.
- [10] Http signature documentation. <https://datatracker.ietf.org/doc/html/draft-cavage-http-signatures>.
- [11] Image on django profile. <https://dev.to/earthcomfy/django-user-profile-3hik>.
- [12] Json documentation. <https://www.w3.org/TR/json-ld11/>.
- [13] Mastodon documentation. <https://docs.joinmastodon.org/>.
- [14] Mastodon security documentation. <https://docs.joinmastodon.org/spec/security/>.
- [15] Security - Mastodon documentation. <https://docs.joinmastodon.org/spec/security/>.
- [16] Webfinger documentation. <https://datatracker.ietf.org/doc/html/rfc7033>.
- [17] Webfinger mastodon documentation. <https://docs.joinmastodon.org/spec/webfinger/>.

- [18] Webfinger w3 documentation. <https://www.w3.org/community/reports/socialcg/CG-FINAL-apwf-20240608/>.
- [19] Emiliano De Cristofaro Nishanth Sastry Aravindh Raman, Sagar Joglekar and Gareth Tyson. Challenges in the decentralised web: The mastodon case. <https://arxiv.org/pdf/1909.05801>, Sep 2019.
- [20] M. Cavage. Signing HTTP messages. <https://datatracker.ietf.org/doc/html/draft-cavage-http-signatures>.
- [21] Paul E. Jones, Gonzalo Salgueiro, Michael B. Jones, and Joseph Smarr. Rfc 7033: Webfinger. <https://datatracker.ietf.org/doc/html/rfc7033>, 2013. Internet Engineering Task Force (IETF).
- [22] Evan Prodromou. *ActivityPub*. O'Reilly Media, Inc., 2024.
- [23] Haris Bin Zia, Jiahui He, Aravindh Raman, Ignacio Castro, Nishanth Sastry, and Gareth Tyson. Flocking to mastodon: Tracking the great twitter migration. <https://arxiv.org/abs/2302.14294>, Feb 2023.